



Data Handling with Python (JSON)

Data Handling with Python (JSON)

In the real world, data is everywhere. Applications store user profiles, e-commerce platforms save product inventories, banking systems maintain transaction records, and social media platforms handle millions of posts. All of this data needs to be stored, transferred, and retrieved in an organized format. One of the most widely adopted formats for this purpose is JSON.

This chapter introduces you to JSON — what it is, why it is used, and how Python makes it simple to work with. By the end of this chapter, you will be able to read data stored in JSON, write data in JSON format, and convert Python objects to and from JSON using Python's built-in json module.

4.1 Introduction to JSON Format

JSON

JSON stands for JavaScript Object Notation. It is a lightweight, text-based format used to store and transfer data. Despite the name, JSON is completely language-independent and is supported by virtually all modern programming languages including Python, Java, JavaScript, C#, and more.

Think of JSON as a universal language for data — no matter what programming language your application is built in, JSON gives everyone a common format to exchange information.

Real-World Analogy — JSON as a Standardized Form

Imagine a college that wants to share a student's academic record with another institution. Rather than each college making up their own file format, they agree to use a common, standardized form with fixed fields: Name, Roll Number, Marks, Courses, etc.

JSON works exactly this way — it provides a standardized structure that every system can understand and use, regardless of the technology they are built on.

4.1.1 What Does JSON Look Like?

JSON organizes data as key-value pairs, very similar to Python dictionaries. Below is an example of a student record stored in JSON format:

```
{
  "name"    : "Riya Mehta",
  "student_id" : "STU-2025-047",
  "age"     : 21,
  "gpa"     : 8.9,
  "is_active" : true,
  "courses" : ["Python", "Data Science", "Statistics"],
  "address"  : {
    "city"   : "Ahmedabad",
    "pincode" : "380015"
  }
}
```

Figure 4.1 — A student record represented in JSON format

Even without any programming knowledge, you can read and understand this data. That is the power of JSON — it is designed to be human-readable and machine-processable at the same time.

4.1.2 JSON Data Types

JSON supports six basic data types. These are the building blocks of any JSON document:

JSON Data Type	Description	Example
String	Text enclosed in double quotes. Always use double quotes — not single.	"Riya Mehta"
Number	Integers or decimals. No quotes around numbers.	21 or 8.9
Boolean	Only two values: true or false. Always lowercase in JSON.	true or false
Null	Represents no value or an empty field.	null
Array	An ordered list of values inside square brackets [].	["Python", "Data Science"]
Object	A collection of key-value pairs inside curly braces { }.	{"city": "Ahmedabad"}

JSON Data Type	Description	Example
 Note — Python vs JSON Data Types Python and JSON look similar but have important differences: • Python uses True / False (capital T and F). JSON uses true / false (all lowercase). • Python uses None. JSON uses null. • JSON strings must always use double quotes (" "). Single quotes are not allowed in JSON.		

4.1.3 Why is JSON So Widely Used?

JSON became the standard data exchange format for a number of practical reasons:

Reason	Explanation
Human-Readable	JSON is written as plain text, making it easy for developers to read and debug.
Lightweight	JSON uses minimal characters compared to formats like XML, reducing file size and transfer time.
Language-Independent	Every major programming language has built-in tools to parse and generate JSON.
Widely Supported	REST APIs, databases (MongoDB), configuration files, and web services all use JSON.
Directly Maps to Data Structures	JSON objects map directly to Python dictionaries, and JSON arrays map to Python lists.

4.2 Reading JSON Data

Before your Python program can use data stored in JSON format, it must read and interpret that data. This process is called parsing. Python's built-in `json` module makes parsing JSON straightforward. There are two primary scenarios for reading JSON data:

- Reading JSON from a string (data already in your program's memory)
- Reading JSON from a file (data stored on disk as a `.json` file)

4.2.1 Reading JSON from a String using `json.loads()`

When JSON data is already present as a Python string — for example, received from a web API — you use the `json.loads()` function. The 's' in `loads` stands for string.

`json.loads()`

Converts a JSON-formatted string into a Python object (typically a dictionary or list). The 's' suffix stands for 'string'.

Example 1 — Reading a Simple JSON String:

```
import json

# JSON data as a string (as if received from a web API)
json_string = '{"name": "Arjun Patel", "age": 22, "city": "Surat"}'

# Parse the JSON string into a Python dictionary
student = json.loads(json_string)

# Now we can access data like a regular Python dictionary
print(student["name"]) # Output: Arjun Patel
print(student["age"])  # Output: 22
print(student["city"]) # Output: Surat
print(type(student))   # Output: <class 'dict'>
```

Example 2 — Reading JSON with a List (Array):

```
import json

# JSON string containing an array of courses
json_string = '{"student": "Priya Shah", "courses": ["Python", "SQL", "Excel"]}'

data = json.loads(json_string)

print(data["student"]) # Output: Priya Shah
print(data["courses"]) # Output: ['Python', 'SQL', 'Excel']
```

```

print(data["courses"][0])    # Output: Python (first item)
print(data["courses"][-1])   # Output: Excel (last item)

# Loop through all courses
for course in data["courses"]:
    print("Enrolled in:", course)

```

4.2.2 Reading JSON from a File using json.load()

In most real-world applications, JSON data is stored in .json files on disk — for example, configuration files, database exports, or API response caches. To read such files, you use `json.load()` (without the 's').

json.load()

Reads JSON data directly from a file object and converts it into a Python object. Used with the `open()` function to access files on disk.

Step 1 — First, let us understand what a .json file looks like. Suppose we have a file named `employee.json` stored on our computer:

```

{
  "employee_id" : "EMP-1042",
  "name"       : "Kavya Nair",
  "department" : "Data Analytics",
  "salary"    : 72000,
  "is_permanent" : true,
  "skills"     : ["Python", "Power BI", "SQL", "Excel"]
}

```

Step 2 — Now write the Python code to read this file:

```

import json

# Open the JSON file and read its contents
with open("employee.json", "r") as file:
    employee = json.load(file)

# Access individual fields
print("Name      :", employee["name"])
print("Department :", employee["department"])
print("Salary    :", employee["salary"])
print("Permanent  :", employee["is_permanent"])

```

```
# Loop through the skills list
print("\nSkills:")
for skill in employee["skills"]:
    print(" -", skill)
```

💡 Pro Tip — json.load() vs json.loads()

json.load(file) → reads from a FILE object (file on disk)

json.loads(string) → reads from a STRING (text in memory)

Memory trick: The extra 's' in loads stands for string. If you are reading from a file, use load. If you are reading from a string, use loads.

Example 3 — Reading a JSON File with a List of Records:

Suppose we have a file called products.json containing an array of product records:

```
[
    {"product_id": "P001", "name": "Laptop", "price": 55000, "in_stock": true},
    {"product_id": "P002", "name": "Mouse", "price": 799, "in_stock": true},
    {"product_id": "P003", "name": "Monitor", "price": 18500, "in_stock": false}
]
```

Figure 4.6 — products.json file with a list of product records

```
import json

# Read the JSON file containing a list of products
with open("products.json", "r") as file:
    products = json.load(file)

# products is now a Python list of dictionaries
print("Total products:", len(products))

# Print each product with formatted output
for product in products:
    status = "Available" if product["in_stock"] else "Out of Stock"
    print(f"{product['name']:15} | Rs. {product['price']:>6} | {status}")
```

Figure 4.7 — Iterating over a JSON array loaded from a file

4.3 Writing JSON Data

Just as we read JSON data into Python, we often need to do the reverse — take Python data and save it as JSON. This is called serialization. You will do this whenever your program needs to save its output, export results, or create a file that another application can read.

4.3.1 Writing JSON to a File using `json.dump()`

`json.dump()`

Converts a Python object into JSON format and writes it directly to a file. The 'dump' without 's' writes to a file object.

Example 1 — Writing a Dictionary to a JSON File:

```
import json

# Python dictionary — student information
student = {
    "name" : "Rohan Verma",
    "roll_no" : "2025CS101",
    "semester" : 4,
    "gpa" : 8.6,
    "is_active" : True,
    "subjects" : ["Algorithms", "DBMS", "Operating Systems"]
}

# Write the dictionary to a JSON file
with open("student_record.json", "w") as file:
    json.dump(student, file, indent=4)

print("Student record saved successfully.")
```

Figure 4.8 — Writing a Python dictionary to a JSON file

The resulting `student_record.json` file will look exactly like this:

```
{
  "name" : "Rohan Verma",
  "roll_no" : "2025CS101",
  "semester" : 4,
  "gpa" : 8.6,
  "is_active" : true,
```

```

"subjects" : [
    "Algorithms",
    "DBMS",
    "Operating Systems"
]
}

```

Figure 4.9 — Output written to `student_record.json` (notice Python `True` became JSON `true`)



Note — The indent Parameter

When using `json.dump()` or `json.dumps()`, the `indent` parameter controls the formatting:

- `json.dump(data, file)` — All on one line, no spaces (compact, hard to read).
- `json.dump(data, file, indent=4)` — Each key-value on a new line, indented by 4 spaces.

Always use `indent=4` when writing files that humans will read. Omit it when optimizing file size.

Example 2 — Writing a List of Records to a JSON File:

```

import json

# List of employee records
employees = [
    {"id": "E001", "name": "Anjali Singh", "dept": "Finance", "salary": 65000},
    {"id": "E002", "name": "Mehul Desai", "dept": "IT", "salary": 82000},
    {"id": "E003", "name": "Pooja Agarwal", "dept": "HR", "salary": 58000},
]

# Save to a JSON file
with open("employees.json", "w") as file:
    json.dump(employees, file, indent=4)

print(f"Saved {len(employees)} employee records to employees.json")

```

Figure 4.10 — Writing a list of dictionaries to a JSON file

4.4 Using the json Module

Python's json module is part of the standard library — this means it comes pre-installed with Python and does not require any additional installation. You simply import it at the top of your file.

```
import json # This single line gives you access to all JSON functions
```

Figure 4.11 — Importing the json module

4.4.1 Overview of json Module Functions

The json module provides four primary functions. Two are for reading (deserializing) and two are for writing (serializing):

Function	Direction	Works With	Description
json.loads()	JSON → Python	String	Parse a JSON string and return a Python object
json.load()	JSON → Python	File	Read a JSON file and return a Python object
json.dumps()	Python → JSON	Returns String	Convert Python object to a JSON-formatted string
json.dump()	Python → JSON	File	Convert and write Python object to a JSON file

4.4.2 How Python and JSON Types Map to Each Other

When Python converts between Python objects and JSON, the following type conversions happen automatically:

Python Type	→ JSON Type	Example
dict	object { }	{"name": "Riya"}
list	array []	["Python", "SQL"]
tuple	array []	["Mon", "Tue", "Wed"]
str	string " "	"Hello"
int	number	42
float	number	8.75
True	true	true
False	false	false
None	null	null



Important —
Tuples Are Converted
to Arrays

Python Type	→ JSON Type	Example
When you convert a Python tuple to JSON using <code>json.dumps()</code>, it becomes a JSON array (list). When you read it back with <code>json.loads()</code>, it comes back as a Python list, not a tuple. This is a one-way conversion. If preserving the tuple type is important, you need to handle the conversion manually after reading the JSON.		

4.5 Converting Python Objects to JSON — `dump()` and `dumps()`

Converting a Python object into JSON format is called serialization or encoding. Python provides two functions for this: `json.dumps()` for generating a JSON string, and `json.dump()` for writing directly to a file. We have already seen `json.dump()` in Section 4.3. This section focuses on `json.dumps()` and the advanced options available in both functions.

4.5.1 `json.dumps()` — Converting to a JSON String

<code>json.dumps()</code>	Converts a Python object (dict, list, etc.) into a JSON-formatted string. The 's' suffix stands for string. The result is returned as a string — it is not written to a file.
----------------------------------	---

Example 1 — Basic Conversion:

```
import json

# A Python dictionary
order = {
```

```

"order_id" : "ORD-8821",
"customer" : "Sneha Joshi",
"amount" : 4250.75,
"items" : ["Keyboard", "USB Hub", "Webcam"],
"delivered" : False
}

# Convert to JSON string
json_string = json.dumps(order)

print(json_string)
print(type(json_string)) # Output: <class 'str'>

```

Example 2 — Formatted (Readable) JSON String:

```

import json

order = {
    "order_id" : "ORD-8821",
    "customer" : "Sneha Joshi",
    "amount" : 4250.75,
    "items" : ["Keyboard", "USB Hub", "Webcam"],
    "delivered" : False
}

# Pretty-print with indentation for readability
json_string = json.dumps(order, indent=4)
print(json_string)

```

4.5.2 Useful Parameters for dump() and dumps()

Both `json.dump()` and `json.dumps()` accept additional parameters that control the output format:

Parameter	Default	Description	Example
indent	None	Number of spaces for indentation. None gives compact output.	indent=4
sort_keys	False	If True, output dictionary keys are sorted alphabetically.	sort_keys=True
separators	(" ", ": ")	Custom separators for items and keys. Use ('', ':') for ultra-compact output.	separators=(',', ':')

Parameter	Default	Description	Example
ensure_ascii	True	If False, allows non-ASCII characters (e.g., Gujarati, Hindi) to appear as-is.	ensure_ascii=False

Example 3 — Sorted Keys and Non-ASCII Characters:

```
import json

# Student record with Hindi name
student = {
    "name" : "Rahul Kumar",
    "city" : "Mumbai",
    "native_name" : "राहुल कुमार",
    "age" : 23,
    "gpa" : 9.1
}

# Sort keys alphabetically, preserve Unicode characters
output = json.dumps(student, indent=4, sort_keys=True, ensure_ascii=False)
print(output)
```

Example 4 — Serializing a List of Dictionaries:

```
import json

# Inventory data as a list of Python dicts
inventory = [
    {"item": "A4 Paper Ream", "qty": 50, "unit_price": 220},
    {"item": "Ballpoint Pens", "qty": 200, "unit_price": 15},
    {"item": "Stapler", "qty": 10, "unit_price": 350},
    {"item": "Sticky Notes", "qty": 30, "unit_price": 85},
]

# Convert the entire list to a JSON string
inventory_json = json.dumps(inventory, indent=4)

# Show a portion of the output
print(inventory_json)

# Count how many items are in the inventory
```

```
total_items = len(json.loads(inventory_json))
print(f"\nTotal inventory categories: {total_items}")
```

4.6 Converting JSON to Python Objects — load() and loads()

Converting JSON data back into Python objects is called deserialization or decoding. This is what allows Python to work with external data — from APIs, databases, configuration files, or any other source that uses JSON. Python provides two functions: `json.loads()` for strings and `json.load()` for files. We covered their basic usage in Section 4.2. This section explores them in greater depth with advanced examples.

4.6.1 Comprehensive Example — Full Round Trip

A round trip means converting Python → JSON → Python and verifying the data is preserved correctly. This is a fundamental concept in data handling:

```
import json

# Step 1: Start with a Python dictionary
original_data = {
    "project" : "LMS Development",
    "team_size" : 8,
    "budget" : 1500000.00,
    "completed" : False,
    "milestones" : ["Planning", "Design", "Development", "Testing"],
    "lead" : {"name": "Vikram Rao", "email": "vikram@company.com"}
}

# Step 2: Convert Python dict → JSON string (Serialization)
json_string = json.dumps(original_data, indent=4)
print("--- JSON String ---")
print(json_string)

# Step 3: Convert JSON string → Python dict (Deserialization)
recovered_data = json.loads(json_string)

# Step 4: Verify the round trip
print("\n--- Verification ---")
print("Project :", recovered_data["project"])
print("Team Lead :", recovered_data["lead"]["name"])
```

```
print("Milestone :", recovered_data["milestones"][2]) # Development
print("Type      :", type(recovered_data))
```

4.6.2 Accessing Nested JSON Data

Real-world JSON data is often deeply nested — objects within objects within arrays. Python makes it straightforward to navigate this structure using chained indexing.

```
import json

# Nested JSON — a company record with departments and employees
json_data = '''
{
  "company"   : "TechBridge Solutions",
  "established" : 2018,
  "departments" : [
    {
      "name"    : "Engineering",
      "head"    : "Amit Shah",
      "employees" : 34
    },
    {
      "name"    : "Data Science",
      "head"    : "Priya Mehta",
      "employees" : 12
    }
  ]
}
'''

company = json.loads(json_data)

# Accessing top-level fields
print("Company   :", company["company"])
print("Since     :", company["established"])

# Accessing the first department
first_dept = company["departments"][0]
print("Dept 1    :", first_dept["name"])
```

```

print("Head    :", first_dept["head"])

# Accessing the second department head directly
print("Dept 2 Head:", company["departments"][1]["head"])

# Looping through all departments
print("\nAll Departments:")
for dept in company["departments"]:
    print(f" {dept['name']:20} | Head: {dept['head']:15} | Staff: {dept['employees']}")

```

4.6.3 Handling Missing Keys Safely

When working with JSON data from external sources, some fields may be missing. Using direct indexing (`data["key"]`) on a missing key raises a `KeyError`. The safe approach is to use the `.get()` method, which returns a default value if the key does not exist:

```

import json

# JSON data — note: "phone" field is missing for one employee
json_string = '''
[
    {"name": "Divya Nair", "email": "divya@co.com", "phone": "9876543210"},
    {"name": "Karan Mehta", "email": "karan@co.com"}
]
'''

employees = json.loads(json_string)

for emp in employees:
    name = emp.get("name", "Unknown")
    email = emp.get("email", "Not provided")
    phone = emp.get("phone", "Not provided") # Safe — returns default if missing
    print(f"Name: {name:15} | Email: {email:22} | Phone: {phone}")

```

4.6.4 Reading and Processing a Real-World JSON File

Let us bring everything together with a complete, practical example. We will read a JSON file containing sales records, process the data, and generate a summary report:

First, assume we have a file called `sales_data.json`:

```
[
    {"month": "January", "region": "North", "sales": 125000, "target": 120000},
    {"month": "January", "region": "South", "sales": 98000, "target": 100000},
    {"month": "February", "region": "North", "sales": 134000, "target": 130000},
    {"month": "February", "region": "South", "sales": 112000, "target": 105000},
    {"month": "March", "region": "North", "sales": 145000, "target": 140000},
    {"month": "March", "region": "South", "sales": 103000, "target": 110000}
]
```

Figure 4.19 — sales_data.json file with monthly regional sales figures

```
import json

# Read the sales data file
with open("sales_data.json", "r") as file:
    sales_records = json.load(file)

print("=" * 60)
print(f" QUARTERLY SALES PERFORMANCE REPORT")
print("=" * 60)
print(f" {"Month":12} {"Region":8} {"Sales":>10} {"Target":>10} {"Status":>10}")
print("-" * 60)

total_sales = 0
total_target = 0

for record in sales_records:
    status = "ACHIEVED" if record["sales"] >= record["target"] else "MISSED"
    print(f" {"record["month"]:12} {"record["region"]:8} '
          f'{"record["sales"]:>10,} {"record["target"]:>10,} {"status:>10}')"
    total_sales += record["sales"]
    total_target += record["target"]

print("=" * 60)
print(f" TOTAL SALES : Rs. {total_sales:,}")
print(f" TOTAL TARGET : Rs. {total_target:,}")
achievement = (total_sales / total_target) * 100
print(f" ACHIEVEMENT : {achievement:.1f}%")
```

Chapter Summary

Chapter Summary

- JSON (JavaScript Object Notation) is a universal, text-based format for storing and exchanging data. It is human-readable, lightweight, and supported by all major programming languages.
- JSON supports six data types: String, Number, Boolean, Null, Array (list), and Object (key-value pairs).
- Python's json module is built-in — no installation needed. Import it with: `import json`
- `json.loads(string)` parses a JSON string into a Python object (dict/list).
- `json.load(file)` reads a JSON file from disk and returns a Python object.
- `json.dumps(object)` converts a Python object into a JSON-formatted string.
- `json.dump(object, file)` converts a Python object and writes it directly to a file.
- The `indent=4` parameter formats JSON output for human readability.
- `sort_keys=True` sorts dictionary keys alphabetically in the output.
- `ensure_ascii=False` preserves Unicode characters such as Hindi, Gujarati, or Chinese text.
- Python True/False/None convert to JSON true/false/null respectively.
- Use the `.get()` method to safely access keys that may not exist in parsed JSON data.
- A 'round trip' (Python → JSON → Python) verifies that data is preserved correctly through serialization and deserialization.

Quick Reference — json Module

Function	Use Case	Code Snippet
<code>json.loads(s)</code>	Parse JSON string → Python object	<code>data = json.loads('{"key": "value"}')</code>
<code>json.load(f)</code>	Read JSON file → Python object	<code>with open("f.json") as f: data = json.load(f)</code>
<code>json.dumps(obj)</code>	Python object → JSON string	<code>text = json.dumps(data, indent=4)</code>
<code>json.dump(obj, f)</code>	Write Python object → JSON file	<code>with open("f.json", "w") as f: json.dump(data, f, indent=4)</code>